Project Report

On

Urban Mobility Optimization Using LLM and GraphRAG

Submitted in partial fulfilment for the award of

Diploma in Big Data Analytics

from C-DAC Hyderabad

Guided by:

Mrs. Amrita Budarapu

Presented by:

Mr. Bhavya Jain	**PRN No. 250850325010**
Mr. Yash Ashok Ambildhuke	**PRN No. 250850325040**
Ms. Mansi Omprakash Waghmare	**PRN No. 250850325039**
Mr. Komuravelli Sai Ashrith	**PRN No. 250850325022**

Centre for Development of Advanced Computing (C-DAC),Hyderabad

ACKNOWLEDGEMENT

This project "**Urban Mobility Optimizer Using LLMs and GraphRAG**" was a great learning experience for us and we are submitting this work to CDAC Hyderabad.

We are very glad to mention the name of *Mrs.Amrita* for her valuable guidance to work on this project. Her guidance and support helped me to overcome various obstacles and intricacies during the course of project work.

We are highly grateful to Mr.Sharanbasappa , Training co-ordinator. C-DAC Hyderabad, for his guidance and support whenever necessary while doing this course Diploma in *Advanced Big Data Analytics (DBDA)* through C-DAC Hyderabad

Our heartfelt thanks goes to *Mr. Sadhu Sreenivas* (Course Coordinator, PG-DBDA) who gave all the required support and kind coordination to provide all the necessities and extra hours to complete the project and throughout the course up to the last day here in C-DAC Hyderabad

From:

Mr. Bhavya Jain

Mr. Yash Ashok Ambildhuke

Ms. Mansi Omprakash Waghmare

Mr.Komuravelli Sai Ashrith

TABLE OF CONTENTS

CONTENTS	Page No
1. Introduction of Project	**4**
2.Product Overview and Summary 2.1Purpose 2.2Scope 2.3Overview 2.4Feasibility Study	**5-7**
3.Overall Description 3.1Product Feature 3.2Technology Used 3.3User Classes 3.4 General Constraints	**8-10**
4.Requirement 4.1Functional Requirement	**11-13**
5.Design 1.1 High Level Design 1.2 Database Design	**14-15**
6.Interface(UI)	**16-17**
7.Project Management Methodology	**18**
8.Test Report	**19-20**
9.Future Scope	**21**

1.INTRODUCTION OF PROJECT:

Urban transportation has become increasingly complex due to rapid population growth, traffic congestion, and the availability of multiple transport modes in modern cities. Selecting the most suitable mode of transportation is no longer limited to identifying the shortest route but also depends on factors such as travel cost, travel time, public transport availability, and environmental conditions.

The Urban Mobility Optimizer Using LLM and GraphRAG is designed to address this challenge by providing an intelligent, data-driven, and explainable travel decision support system. The project focuses on Hyderabad city, where transport options such as cars, buses, and metro services operate within a dense and highly connected road network.

Unlike traditional navigation systems that primarily display routes or distances, this system analyzes multiple real-world parameters and provides clear explanations for its recommendations. By combining graph databases (Neo4j), Large Language Models (LLMs), and Graph Retrieval-Augmented Generation (GraphRAG), the system delivers accurate, context-aware, and easily understandable mobility recommendations.

2. PRODUCT OVERVIEW AND SUMMARY

2.1 Purpose:

The primary purpose of the Urban Mobility Optimizer Using LLM and GraphRAG is to assist urban commuters in selecting the most suitable mode of transportation based on multiple real-world factors rather than relying only on distance or route length. In cities like Hyderabad, travelers often face challenges such as traffic congestion, varying transport availability, fluctuating travel costs, and changing weather conditions. This project aims to simplify travel decision-making by analyzing these factors together and providing clear, data-driven recommendations that help users make informed choices.

Another key objective of the project is to introduce explainability into urban mobility systems. Traditional navigation applications typically provide route suggestions without explaining why a particular option is recommended. This project addresses that gap by using Large Language Models (LLMs) combined with Graph Retrieval-Augmented Generation (GraphRAG) to generate natural-language explanations grounded in actual city graph data. This ensures that users not only receive recommendations but also understand the reasoning behind them, increasing trust and usability.

Additionally, the project aims to demonstrate the practical application of agent-based AI architecture in solving real-world problems. By dividing tasks such as route calculation, weather analysis, cost estimation, transport availability checking, and explanation generation into independent agents coordinated by an orchestrator, the system becomes modular, scalable, and intelligent. Overall, the project showcases how modern AI techniques, graph databases, and LLMs can be effectively combined to build a smart, explainable, and user-friendly urban mobility solution.

2.2 Scope

The scope of the Urban Mobility Optimizer Using LLM and GraphRAG is limited to analyzing and recommending urban transportation options within Hyderabad city. The system focuses on commonly used modes of transport such as car, bus, and metro, and evaluates them based on key factors including travel distance, estimated travel time, cost, weather conditions, and transport availability. The project uses real road and transport connectivity data to ensure that the recommendations are realistic and relevant to actual urban travel scenarios.

This project covers decision support and explainable recommendations rather than real-time navigation or live traffic rerouting. It does not aim to replace full-scale navigation applications but instead acts as an intelligent assistant that helps users decide which transport mode is most suitable for their journey. The system is designed for short to medium urban trips and demonstrates how AI and graph-based data can support everyday mobility decisions.

The project also focuses on showcasing modern AI concepts such as agent-based architecture, graph databases, and LLM-driven explanations. However, it does not include features such as ticket booking, fare payment, or live vehicle tracking. Future enhancements may include real-time traffic data, additional transport modes, and expansion to other cities.

2.3 Overview

The Urban Mobility Optimizer Using LLM and GraphRAG is an intelligent travel decision support system that helps users choose the best transportation option for their journey within

Hyderabad. The system takes the source and destination provided by the user and analyzes multiple factors such as route distance, estimated travel time, cost of travel, weather conditions, and the availability of public transport.

The project uses a Neo4j graph database to model the city's road network and transport connectivity. Graph queries are used to calculate routes and determine whether bus or metro services are available near the selected locations. Weather data is fetched from an external API to make the analysis more realistic. Each task is handled by a specialized AI agent, and an orchestrator agent coordinates these agents to produce the final result.

A major highlight of the system is the use of Large Language Models (LLMs) combined with GraphRAG. Relevant graph data is retrieved and structured, and the LLM generates a clear, user-friendly explanation that is grounded in actual city data. The final output is presented through an interactive Streamlit dashboard, where users can view cost and time comparisons, charts, route maps, and detailed explanations. This project demonstrates how AI, graph databases, and explainable systems can work together to solve real-world urban mobility problems.

2.4 Feasibility Study

A feasibility study was conducted to evaluate whether the Urban Mobility Optimizer Using LLM and GraphRAG can be successfully developed and used in a real-world environment. The study focuses on technical and operational aspects to ensure that the system is practical, reliable, and suitable for urban mobility decision support. Based on the analysis, the project is feasible as it uses existing technologies, publicly available data sources, and a modular system design that can be implemented without high infrastructure costs.

1. **Technical Feasibility.**
2. **Operational Feasibility.**

Technical Feasibility

The project is technically feasible as it is built using widely used and well-supported technologies such as Python, Neo4j, Streamlit, and Large Language Models. The road network and transport connectivity are stored and processed efficiently using a graph database, which is suitable for route and connectivity analysis. Weather data is obtained through freely available APIs, and the LLM is integrated locally using Ollama, reducing dependency on paid cloud services. The agent-based architecture allows individual components to work independently, making the system easy to develop, test, and extend. Therefore, the required hardware and software resources are easily accessible and manageable.

Operational Feasibility

The system is operationally feasible because it is designed to be simple and user-friendly. Users can interact with the system through an intuitive Streamlit dashboard where they only need to select source and destination locations. The output is presented in a clear format with tables, charts, maps, and natural language explanations, making it easy for users to understand and use the recommendations. The system does not require specialized technical knowledge to operate, and minimal training is needed. This makes the project suitable for students, researchers, and general users interested in understanding urban travel decisions.

3.OVERALL DESCRIPTION

The Urban Mobility Optimizer Using LLM and GraphRAG is an intelligent decision support system that helps users select the most suitable mode of transportation within an urban environment. The system integrates real-world city data, weather information, and transport availability to analyze travel options such as car, bus, and metro. Instead of only showing routes, the system provides clear explanations and comparisons to support informed travel decisions. The application is designed for Hyderabad city but can be extended to other cities with similar data availability.

3.1Product Features

- Allows users to select source and destination locations within the city
- Calculates travel distance using a city road network graph.
- Estimates travel cost and time for car, bus, and metro
- Checks real-time availability of public transport modes
- Fetches live weather data to enhance decision accuracy
- Generates easy-to-understand explanations using LLM and GraphRAG
- Displays results through tables,charts and interactive maps
- Uses an agent-based architecture for modular and efficient processing

3.2Technology Used

- Python for system development and agent implementation
- Neo4j graph database for modeling road networks and transport connectivity
- GraphRAG for retrieving graph-based context for explanations
- Large Language Model (LLM – Gemma via Ollama) for natural language generation
- Streamlit for building the interactive user interface
- OpenStreetMap / OSMnx for road network data
- Open-Meteo API for real-time weather information
- Matplotlib for cost and travel time visualization

Back End

The backend of the Urban Mobility Optimizer Using LLM and GraphRAG is responsible for processing user requests, performing analysis, and generating intelligent recommendations. It is developed using Python and follows a modular, agent-based architecture. Each agent performs a specific task such as route calculation, weather analysis, cost estimation, transport availability checking, and explanation generation.

The backend uses Neo4j as a graph database to store and query the city's road network and transport connectivity. Graph-based queries are used to calculate distances, check public transport availability, and retrieve contextual information required for GraphRAG. Real-time weather data is fetched using external APIs to improve the accuracy of recommendations.

An Orchestrator Agent coordinates all backend agents, ensuring that data flows correctly between them and that the final output is consistent and complete. The Large Language Model (Gemma via Ollama) is used to generate natural -language explanations based on structured graph data,making the system both intelligent and explainable.

Front End

The frontend of the system is built using Streamlit, which provides an interactive and user-friendly interface. Users can select source and destination locations through dropdown menus and initiate route analysis with a single click. The interface displays important metrics such as distance, weather conditions, travel cost, and estimated travel time.

The frontend also presents comparison tables, bar and line charts, and an interactive map that shows the selected route. Visual indicators help users quickly understand which transport option is more suitable based on cost or time.

3.3User Classes

- General Users: Individuals looking for a better understanding of travel options based on cost, time, and availability
- Students and Researchers: Users studying urban mobility, AI, graph databases, or agent-based systems
- Developers**:** Individuals interested in learning or extending agentic AI and GraphRAG-based applications

3.4General Constraints

- The system is limited to the availability and accuracy of public data sources such as OpenStreetMap and weather APIs
- Real-time traffic conditions are not fully considered in the current version
- Public transport schedules are assumed based on availability rather than live tracking
- The system is currently focused on Hyderabad city only
- Performance depends on local system resources when running the LLM and Neo4j database

4.REQUIREMENTS

4.1 FUNCTIONAL REQUIREMENTS

1. Route Distance Calculation

The system will calculate the shortest route between a selected source and destination using graph-based road network data. The route calculation will be performed using Neo4j graph queries, ensuring accurate distance computation based on real city road connections.

2. Weather Data Retrieval

The system will fetch real-time weather information such as temperature and wind speed for the selected route area using a weather API. This data will be used to enhance travel recommendations and provide realistic context for route planning.

3. Travel Cost Estimation

The system will estimate the travel cost for different transportation modes including car, bus, and metro. Cost calculations will be based on distance, predefined fare models, and transport-specific parameters to ensure practical and understandable results.

4. Travel Time Estimation

The system will calculate the estimated travel time for each available transport mode. Time estimation will consider average speed assumptions and route distance, allowing users to compare faster and slower options effectively.

5. Transport Availability Detection

The system will detect the availability of public transport options such as buses and metro stations using graph-based queries. Only available transport modes will be considered during recommendation and explanation generation.

6. GraphRAG Context Retrieval

The system will retrieve relevant city graph context from the Neo4j database, including nearby road segments, transport nodes, and connectivity details. This contextual data will be structured and passed to the language model to ensure grounded explanations.

7. AI-Based Explanation Generation

The system will generate clear, natural-language explanations using a Large Language Model (LLM) combined with GraphRAG. The explanation will justify the recommended transport option by considering cost, time, weather, and availability factors.

8. Agent Orchestration and Coordination

The system will follow a fully agent-based architecture where different agents handle route calculation, weather analysis, cost estimation, transport checking, and explanation generation. An orchestrator agent will coordinate all agents to ensure smooth and sequential execution.

9. User Interaction and Visualization

The system will provide an interactive web-based dashboard using Streamlit. Users will be able to select source and destination locations, view cost and time comparisons, analyze charts, visualize routes on a map, and read AI-generated explanations.

1.SOFTWARE REQUIREMENTS

1. Operating System

- Windows 10 / Windows 11
- Linux (Ubuntu 20.04 or higher)

2. Programming Language
   - Python 3.9 or higher
3. Web Framework
   - Streamlit (for building the user interface)
4. Graph Database
   - Neo4j (for storing road network and transport connectivity data)
5. Large Language Model Platform
   - Ollama (for running the local LLM such as Gemma)
6. Libraries and Tools
   - Pandas (data processing)
   - Matplotlib (charts and visualization)
   - Requests (API calls for weather data)
   - OSMnx / NetworkX (road network and shortest path calculation)
   - Neo4j Python Driver (graph database connectivity)
7. APIs Used
   - Open-Meteo API (for real-time weather data)
   - OpenStreetMap (for road network data)
8. Development Tools
   - Visual Studio Code / PyCharm
   - Git and GitHub (version control)

2.HARDWARE REQUIREMENTS

1. Processor
   - Minimum: Intel i5 or equivalent
   - Recommended: Intel i7 or higher for better performance
2. RAM
   - Minimum: 8 GB

- Recommended: 16 GB for smooth LLM execution

3. Storage
    - Minimum: 20 GB free disk space
4. Internet Connection
    - Required for fetching weather data and map data
5. System Architecture
    - 64-bit system requi

5.DESIGN

1.System Architecture Overview:

Route Analysis Module:
This module is responsible for calculating the optimal route between the selected source and destination locations. It uses graph-based algorithms on the city road network stored in Neo4j to compute the shortest and most efficient path based on distance and connectivity.

Weather Analysis Module:
This component fetches real-time weather data using external weather APIs. It analyzes factors such as temperature and wind speed, which can influence travel comfort and transport suitability, and passes this information to the decision-making pipeline.

Cost and Travel Time Estimation Module:
This module estimates the travel cost and expected travel time for different transport modes such as car, bus, and metro. The estimation is based on distance, predefined cost models, and average travel speeds for each mode.

Transport Availability Module:
This module checks the availability of public transport options like buses and metro stations along the selected route. It performs spatial and graph queries on the Neo4j database to determine whether these services are accessible for the given journey.

GraphRAG Context Retrieval Module:
This module retrieves relevant contextual data from the graph database, including route details, connectivity information, and transport availability. The retrieved graph data is structured and passed to the LLM to ensure responses are grounded in real city data.

LLM-Based Explanation Module:
Powered by a Large Language Model (LLM), this module generates clear and user-friendly explanations for the recommended transport option. The explanations consider cost, travel time, weather conditions, and availability, making the system's decision transparent and understandable.

Orchestrator and Agent Management Module:
This module acts as the central coordinator of the system. It manages multiple specialized agents such as route agent, weather agent, cost agent, transport agent, and explanation agent, ensuring smooth data flow and task execution across the system.

User Interaction Module:
Implemented using Streamlit, this module allows users to input source and destination locations, view route analysis, compare transport options, analyze charts, and visualize the route on a map through an interactive web interface.

2.Technologies Used:

Programming Language:
Python (used for backend logic, agent implementation, graph queries, and data processing).

Graph Database:
Neo4j for modeling the city road network and transport connectivity.

Large Language Model:
Local LLM (Gemma via Ollama) for generating natural language explanations.

Graph Retrieval-Augmented Generation:
GraphRAG for retrieving structured graph context and grounding LLM responses in real data.

Frontend Framework:
Streamlit for building the interactive user dashboard.

APIs and Libraries:
OSMnx for road network extraction
Open-Meteo API for weather data
Matplotlib for data visualization

Deployment Environment:
Local or cloud-based Python environment with Neo4j and Ollama services running.

3.Component Interaction:

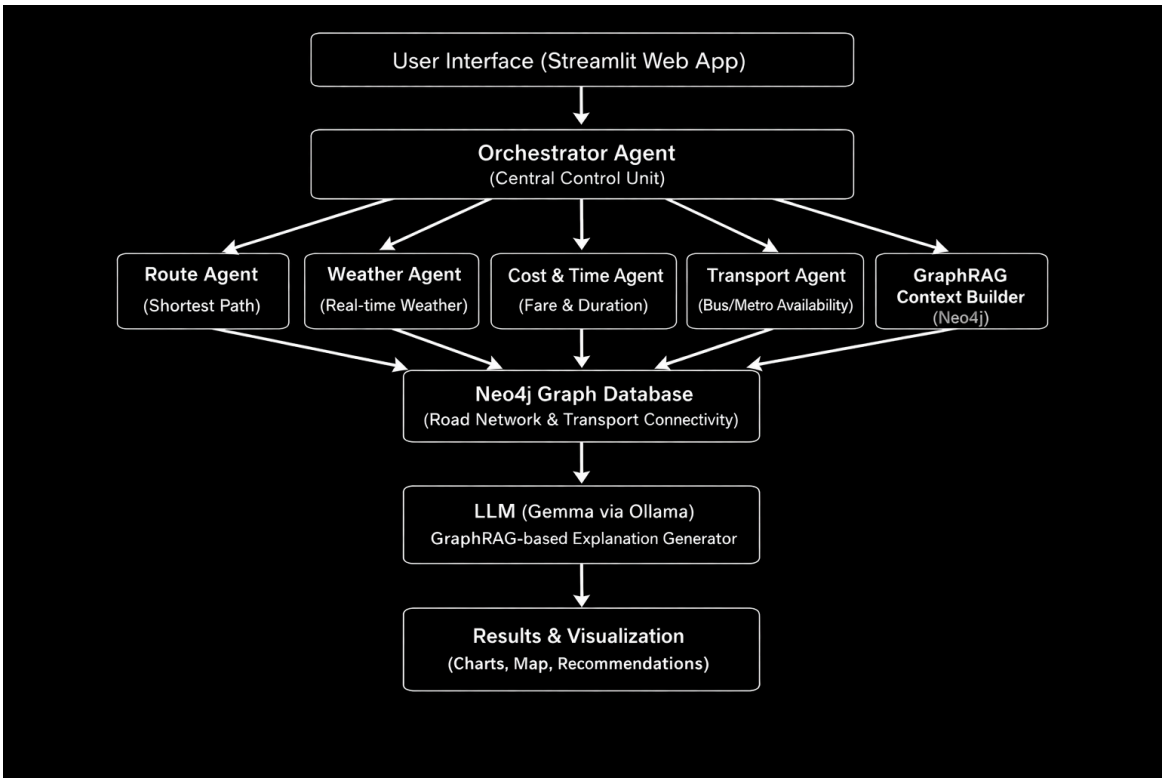
User Input Processing:
Users select source and destination locations through the web-based interface.

Route and Data Analysis:
The route computation module calculates distance and connectivity, while weather, cost, and transport availability modules analyze relevant travel factors.

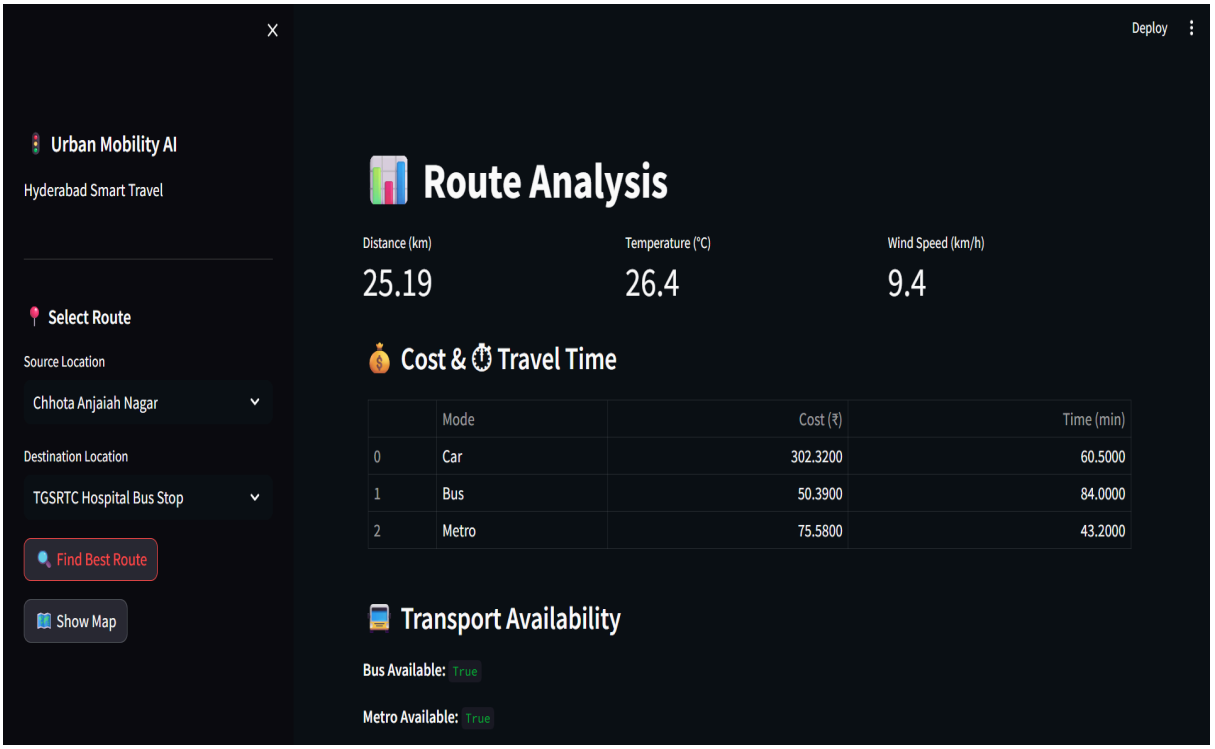
Graph Context Retrieval:
Relevant city graph data is retrieved from Neo4j and structured by the GraphRAG module.

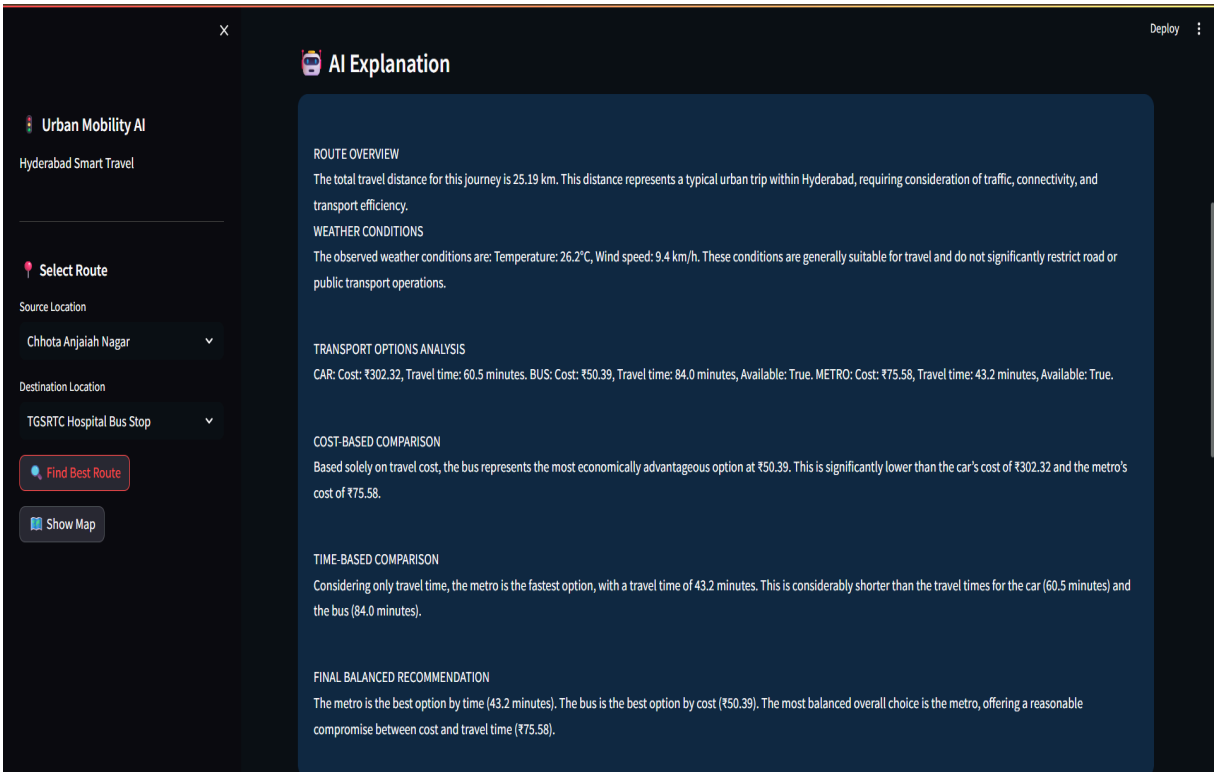
Explanation Generation:
The LLM generates explainable and context-aware transport recommendations based on analytical results and graph data.

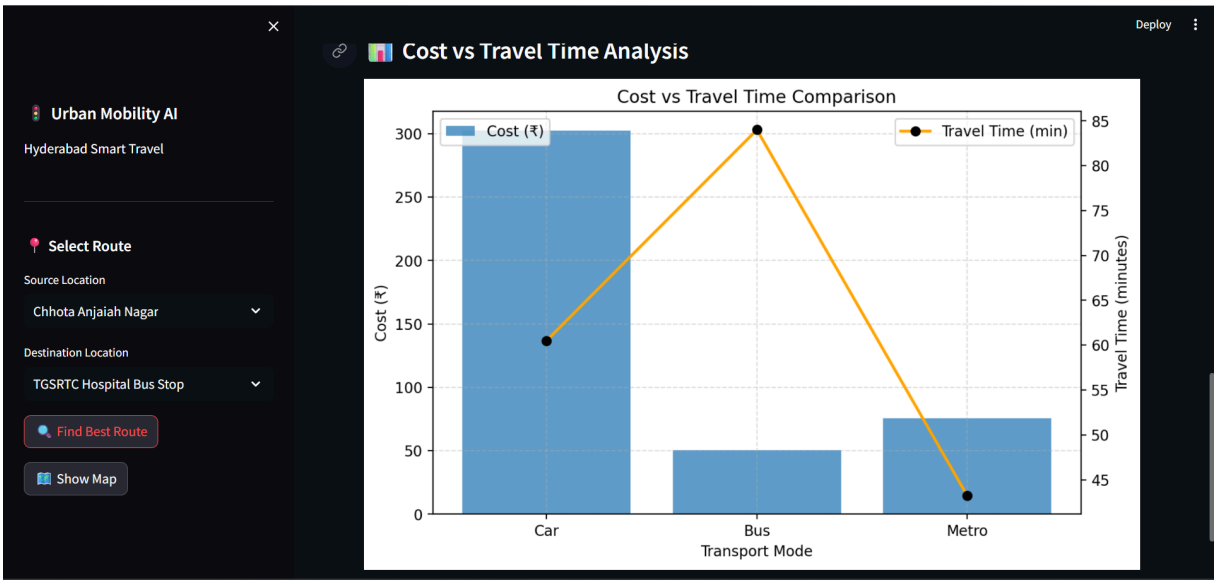
Result Presentation:
Users view travel recommendations, cost and time comparisons, charts, and route maps through the interactive dashboard.



6.USER INTERFACE






Urban Mobility AI
Hyderabad Smart Travel
Select Route
Source Location
Chhota Anjaiah Nagar
Destination Location
TGSRTC Hospital Bus Stop
Find Best Route
Show Map
Deploy
Cost vs Travel Time Analysis
Cost vs Travel Time Comparison
Cost (₹)
Travel Time (min)
Cost (₹)
Travel Time (minutes)
Car
Bus
Metro
Transport Mode

7.PROJECT MANAGEMENT METHODOLOGY

For this project, the Scrum framework, which is a part of Agile methodology, was adopted. Scrum follows an iterative and incremental development approach, making it suitable for complex and data-driven systems like the Urban Mobility Optimizer using LLM and GraphRAG. The project required continuous integration of multiple components such as graph-based routing, weather analysis, transport availability detection, and AI-based explanation generation. Scrum helped in breaking the project into smaller sprints, enabling regular testing, improvements, and feedback-driven development.

Scrum Process and Key Phases

Sprint 0: Project Initialization and Planning

In Sprint 0, the project requirements were analyzed and the overall system architecture was planned. The development environment was set up, including Python, Streamlit, Neo4j Graph Database, and Ollama for running the LLM (Gemma). The initial Product Backlog was created, listing key features such as route calculation using graph data, weather data integration, cost and time estimation, transport availability detection, and AI-based explanation using GraphRAG.

Sprint 1: Core System Development

Sprint 1 focused on implementing the core functionalities of the system. This included modeling the city road network in Neo4j, calculating route distance using graph queries, and integrating real-time weather data APIs. Basic cost and travel time estimation for car, bus, and metro were also implemented during this phase.

Sprint 2: Agent-Based Architecture and UI Development

In Sprint 2, the agent-based architecture was developed. Separate agents were created for route calculation, weather analysis, cost and time estimation, transport availability checking, and GraphRAG context building. The Orchestrator Agent was implemented to coordinate all agents. In parallel, the Streamlit user interface was designed to allow users to select source and destination locations and view results in a structured format.

Sprint 3: GraphRAG Integration and Explanation Generation

Sprint 3 focused on integrating Graph Retrieval-Augmented Generation (GraphRAG) with the Large Language Model. Relevant city graph context was retrieved from Neo4j and passed to the LLM to generate clear, data-grounded explanations. This ensured that the system provided not only recommendations but also understandable reasoning behind each travel option.

Sprint 4: Testing, Optimization, and Final Enhancements

The final sprint involved extensive testing of all system components, including route calculation, weather data retrieval, cost estimation, transport availability detection, and AI explanation generation. Performance optimization was carried out to reduce response time. Final UI improvements were made, and user feedback was incorporated to ensure a smooth and reliable user experience.

8.TEST REPORT:

This section describes the testing objectives, testing methodology, and test cases carried out for the Urban Mobility Optimizer using LLM and GraphRAG. The purpose of testing is to verify that all system components function correctly and work together to provide accurate, reliable, and explainable transportation recommendations. The functionalities tested include route calculation, weather data retrieval, cost and travel time estimation, transport availability detection, GraphRAG-based explanation generation, and user interface interactions.

Testing Objectives

The main objectives of testing are to ensure accurate route distance calculation using the city road graph, correct retrieval of real-time weather data, and reliable estimation of travel cost and time for different transport modes such as car, bus, and metro. Testing also aims to validate the detection of public transport availability using graph queries and to ensure that the AI-generated explanations are clear, relevant, and grounded in actual graph data.

Additionally, testing is conducted to evaluate system performance, responsiveness, scalability, and overall user experience through the Streamlit interface. The robustness of the system under different route conditions and varying data inputs is also assessed.

Test Methodology

The testing process follows multiple standard testing approaches:

Unit Testing focuses on individual components such as route calculation, weather data fetching, cost estimation, transport availability checking, and explanation generation.

Integration Testing ensures that all agents (Route Agent, Weather Agent, Cost Agent, Transport Agent, and GraphRAG module) work together seamlessly under the Orchestrator Agent.

Performance Testing evaluates the system's response time for route analysis and AI explanation generation.

User Acceptance Testing (UAT) verifies that the system meets user expectations and provides an easy-to-use and informative interface.

Security Testing checks the safe handling of API calls and database access, ensuring system stability and data protection.

Test Cases

The following test cases were designed to validate the core functionalities of the system and ensure reliable performance.

TEST CASES:

Test Case	Description	Expected Result	Status
Route Distance Calculation	Test shortest path computation between selected source and destination	Correct route distance is calculated using graph data	Pass
Weather Data Retrieval	Test real-time weather fetching using API	Accurate temperature and wind speed data are displayed	Pass
Cost Estimation	Test travel cost calculation for car, bus, and metro	Correct cost is estimated based on distance	Pass
Travel Time Estimation	Test estimated travel time for each transport mode	Accurate travel time is displayed	Pass
Transport Availability Detection	Test availability of bus and metro using graph queries	Correct availability status is shown	Pass
GraphRAG Context Retrieval	Test retrieval of relevant graph context from Neo4j	Relevant city and transport data is retrieved	Pass
LLM Explanation Generation	Test AI-generated explanation using LLM and GraphRAG	Clear, relevant, and explainable recommendation is generated	Pass
Orchestrator Agent Flow	Test coordination between all agents	All agents execute in correct sequence	Pass
Performance Test	Test system response time for route analysis	System responds within acceptable time	Pass
Scalability Test	Test system behavior with different routes and inputs	System remains stable and responsive	Pass
User Interface Usability	Test Streamlit UI interactions	UI is intuitive and user-friendly	Pass
Visualization Test	Test charts and map rendering	Graphs and maps display correctly	Pass
Security Test	Test API access and database safety	No unauthorized access or failures	Pass

9.FUTURE SCOPE:

1. In the future, the system can be enhanced by integrating live traffic data and real-time congestion updates from traffic APIs. This will allow the Urban Mobility Optimizer to provide more accurate travel time predictions and dynamically adjust route recommendations based on current road conditions.

2. A voice based interface can be added so users can interact with the system hands-free. This feature will allow users to select source and destination locations and receive mobility recommendations through voice commands, improving usability and accessibility.

3.The project can be extended to include environmental impact analysis, , such as estimating carbon emissions for each transport mode. This will help users make eco-friendly travel decisions by comparing sustainability along with cost and travel time.

4.In future versions, the system can introduce personalized travel recommendations and alerts. . Users can receive notifications about delays, transport availability changes, or preferred travel options directly through the dashboard or email based on their travel patterns.